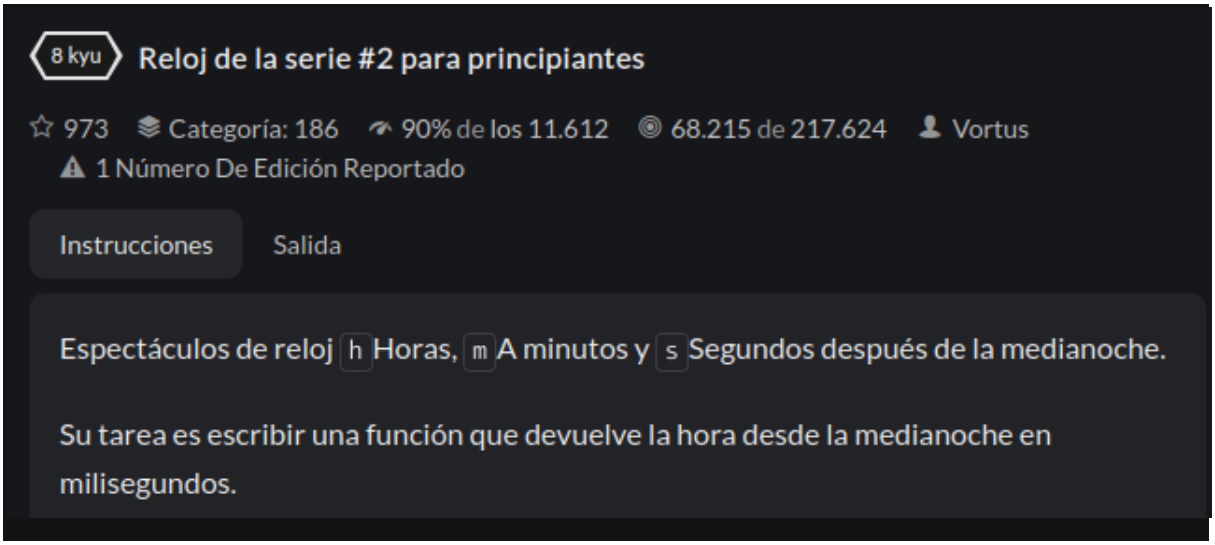
UD00 part 3

Actividad 1 1
Actividad 2 2
Actividad 3 3
Actividad 4 4
Actividad 5 5
Actividad 6 7
Actividad 7 9
Actividad 8 10
Actividad 9 11
Actividad 10 12
Actividad 11 13
Actividad 12 14
Actividad 13 16
Actividad 14 17
Actividad 15 18

Actividad 1




```
def past(h, m, s):
    '''Convierte las horas, minutos y segundos a milisegundos

    PARAMETERS
    ----------
    h (integer):
        Horas a convertir
    m (integer):
        Minutos a convertir
```

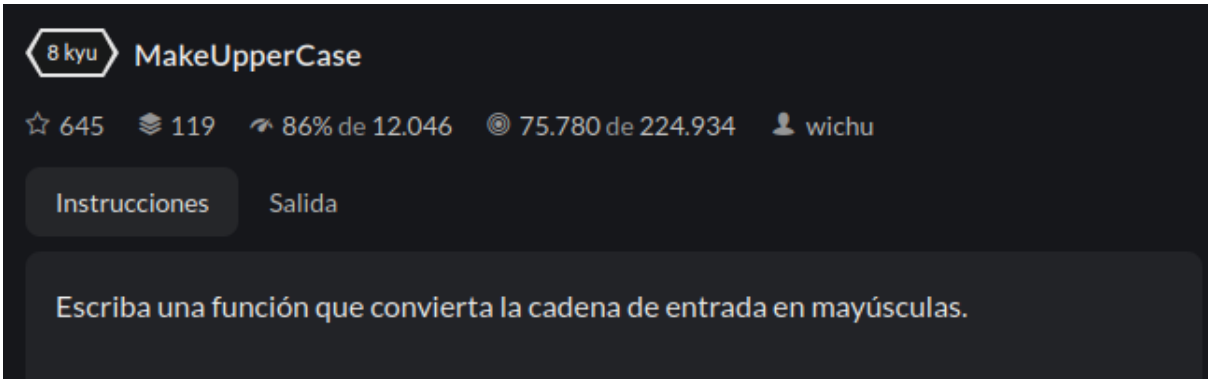
```
    s (integer):
        Segundos a convertir

    RETURNS
    -------
    integer:
        Resultado de la conversión total expresada en milisegundos
    '''
    resultado = (h * 3600000) + (m * 60000) + (s * 1000)
    return resultado

print(past(0, 1, 1))
```

```
ignglitar@ORJ1:~/Escritorio/Python$ /usr/bin/python3 /home/ignglitar/Escritorio/Python/actCodewars1.py
61000
```

Actividad 2




```
def make_upper_case(s):
    '''Convierte una cadena de texto a mayúsculas

    PARAMETERS
    ----------
    s (string):
        Cadena de texto a convertir

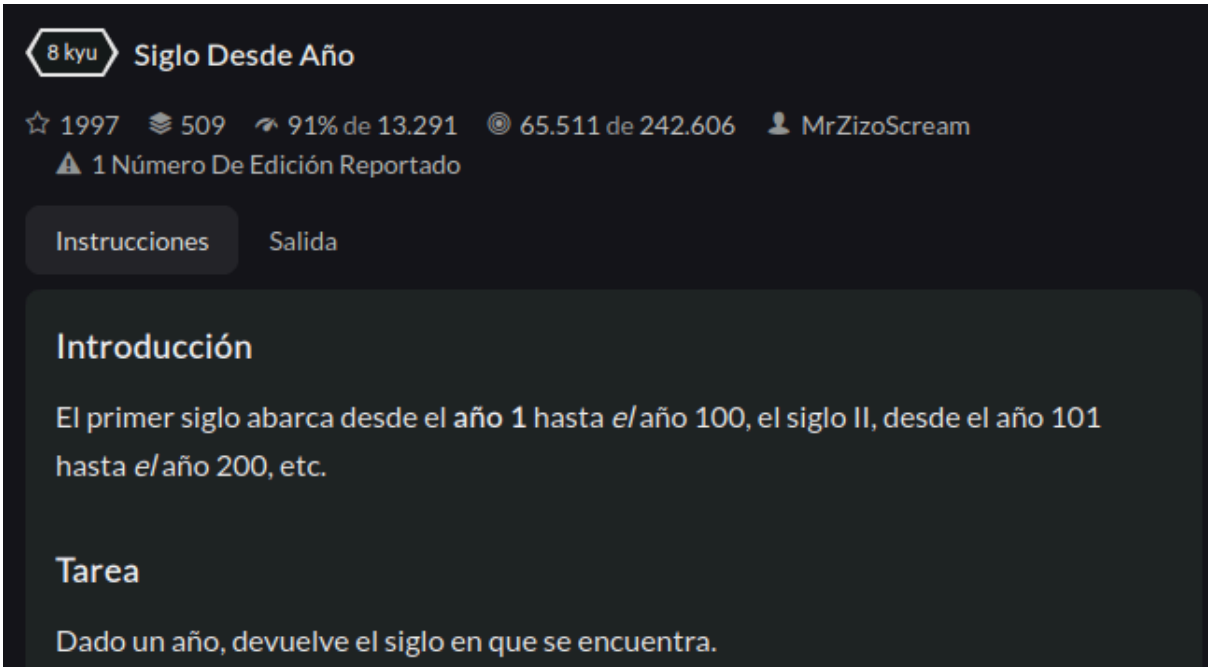
    RETURNS
    -------
    string:
        Cadena de texto convertida completamente a mayúsculas
    '''
    # Convertimos la cadena de texto a mayúsculas
    # Devolvemos el resultado obtenido
    resultado = s.upper()
```

```
    return resultado

print(make_upper_case("nacho"))
```

```
ignglitar@ORJ1:~/Escritorio/Python$ /usr/bin/python3 /home/ignglitar/Escritorio/Python/actCodewars2.py
NACHO
```

Actividad 3




```
def century(year):
    '''Calcula el siglo correspondiente a un año dado

    PARAMETERS
    ----------
    year (integer):
        Año del cual se quiere calcular el siglo

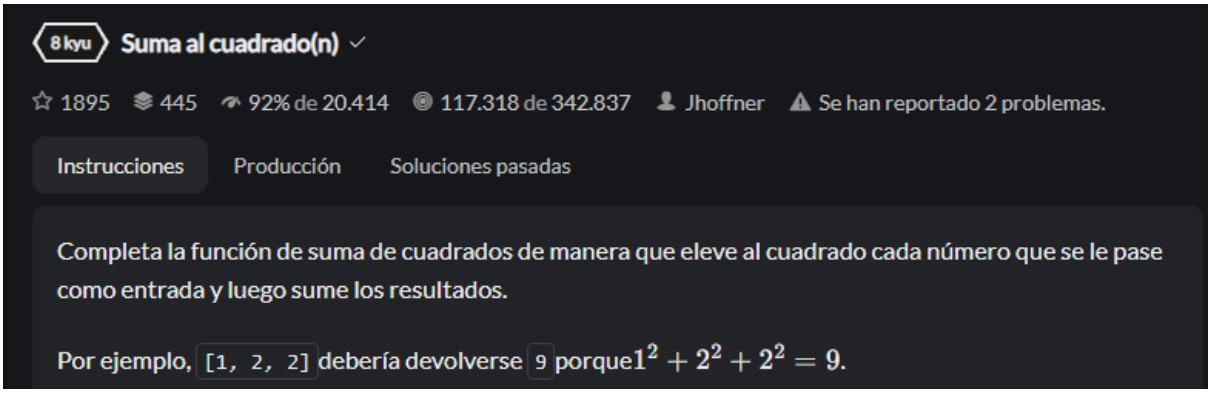
    RETURNS
    -------
    integer:
        Siglo correspondiente al año introducido
    '''
    # Calculamos el siglo sumando 99 al año y dividiendo el resultado
entero entre 100
    # Devolvemos el resultado obtenido
    resultado = (year + 99) // 100
```

```
    return resultado

year = int(input("Introduce el año: "))
print(century(year))
```

```
ignguiltar@ORJ1:~/Escritorio/Python$ /usr/bin/python3 /home/ignguiltar/Escritorio/Python/actCodewars3.py
Introduce tu numero: 1900
19
```

Actividad 4




```
def square_sum(numbers):
    '''Calcula la suma de los cuadrados de una lista de números

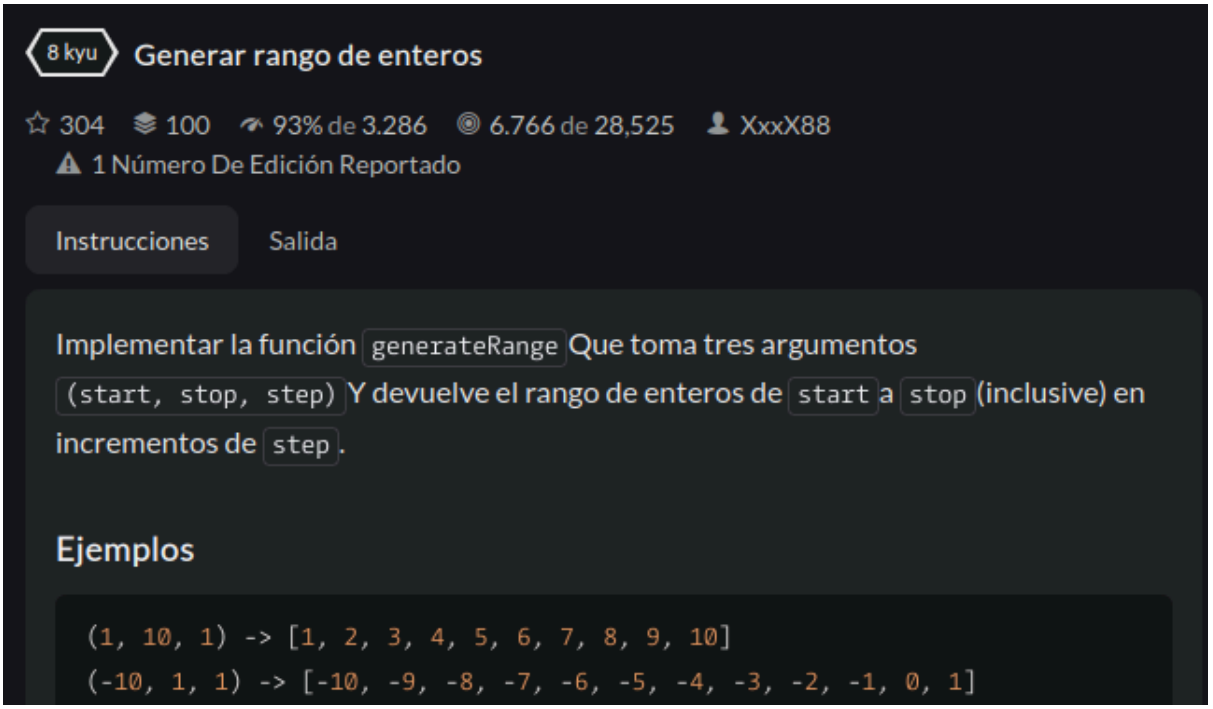
    PARAMETERS
    ----------
    numbers (list):
        Lista de números cuyos cuadrados se van a sumar

    RETURNS
    -------
    integer:
        Suma total de los cuadrados de los números
    '''
    # Inicializamos la variable en 0 para almacenar la suma
    resultado = 0
    # Recorremos cada número de la lista proporcionada
    for numero in numbers:
        # Multiplicamos el número por sí mismo (al cuadrado) y lo
sumamos al resultado acumulado
        resultado = resultado + (numero * numero)
    # Devolvemos el resultado final de la suma
    return resultado
```

```
print(square_sum([1, 2, 2]))
```

```
ignglltar@ORJ1:~/Escritorio/Python$ /usr/bin/python3 /home/ignglltar/Escritorio/Python/actCodewars4.py
9
```

Actividad 5




```
def generate_range(start, stop, step):
    '''Genera una lista de números en un rango determinado con un paso
específico

    PARAMETERS
    ----------
    start (integer):
        Valor inicial del rango
    stop (integer):
        Valor final (límite) del rango
    step (integer):
        Incremento entre cada número del rango

    RETURNS
    -------
    list:
        Lista con los números generados dentro del rango establecido
    '''
```

```
    # Inicializamos una lista vacía para almacenar los números generados
    numeros = []

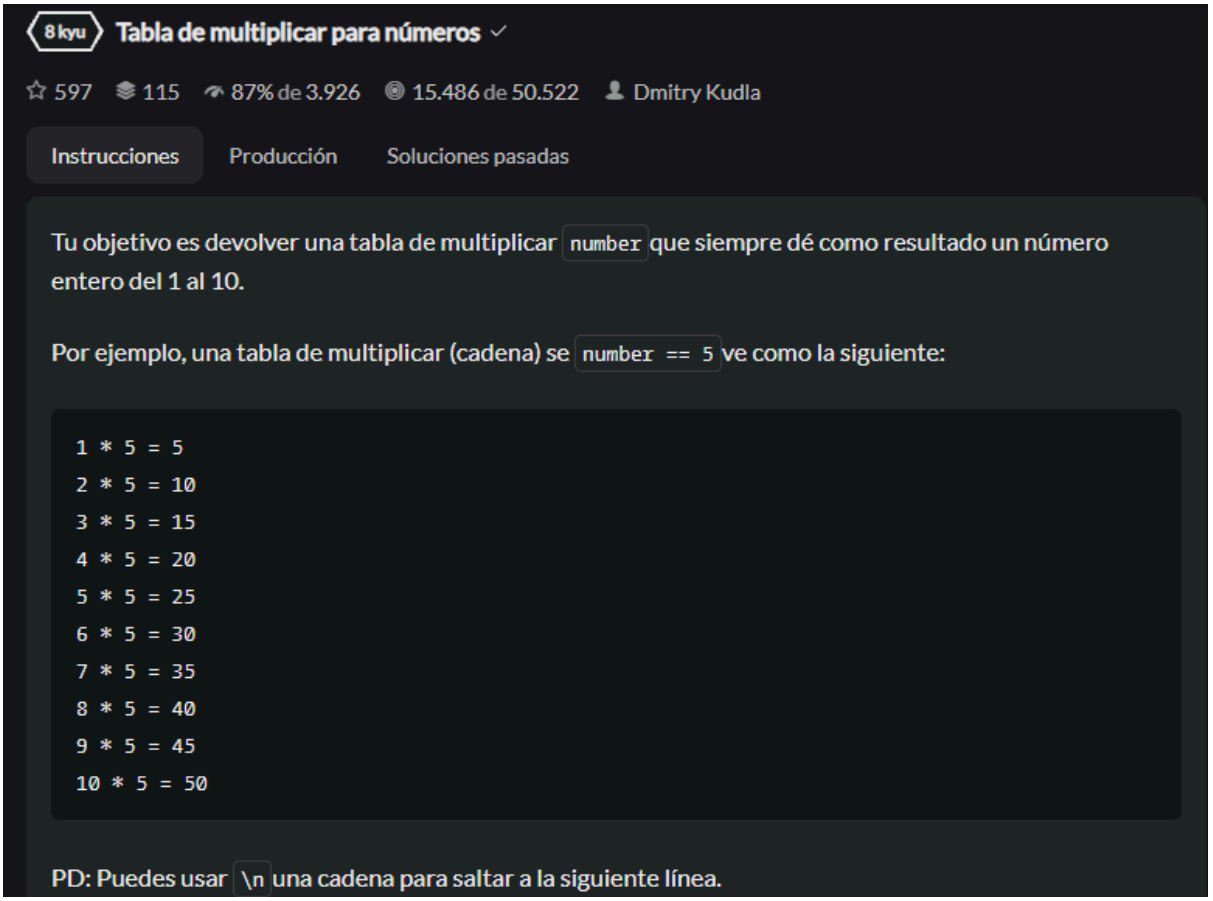
    # Recorremos mientras el valor de inicio sea menor o igual al límite
final
    while start <= stop:
        # Añadimos el valor actual de start a la lista
        numeros.append(start)
        # Incrementamos el valor de start sumándole el paso
correspondiente
        start = start + step

    # Devolvemos la lista completa de números generados
    return numeros

nums = generate_range(1, 10, 2)
print(nums)
```

```
ignguiltar@ORJ1:~/Escritorio/Python$ /usr/bin/python3 /home/ignguiltar/Escritorio/Python/actCodewars5.py
[1, 3, 5, 7, 9]
```

Actividad 6



```python
def multi_table(number):
    '''Genera la tabla de multiplicar de un número dado del 1 al 10 en formato de texto

    PARAMETERS
    ----------
    number (integer):
        Número del cual se generará la tabla de multiplicar

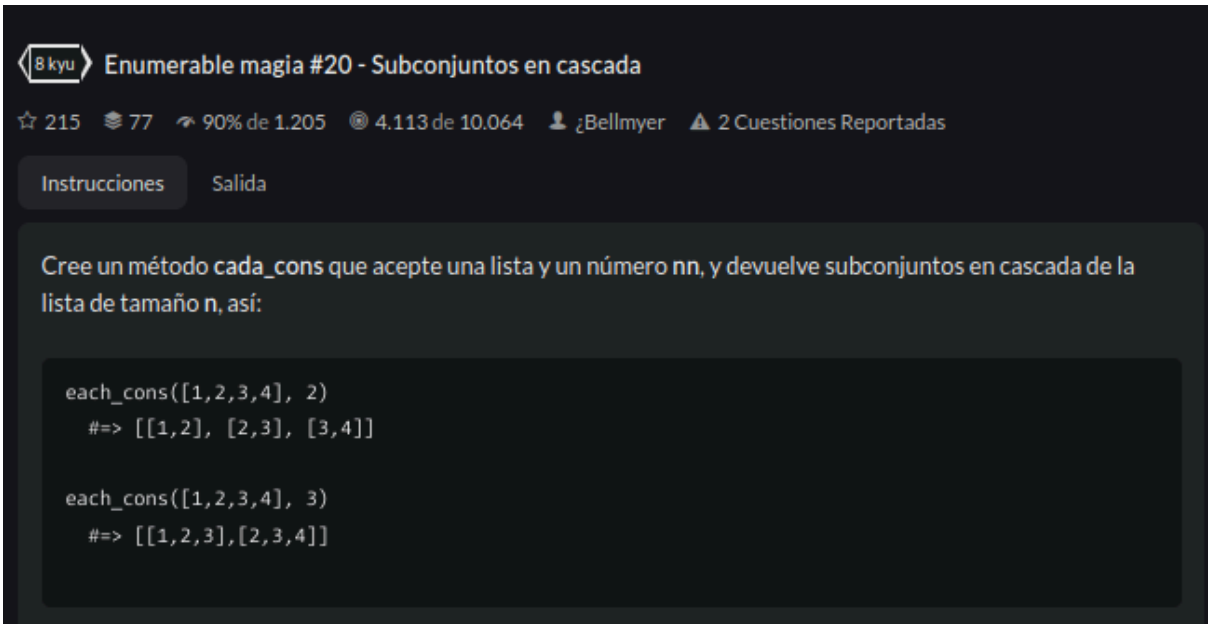
    RETURNS
    -------
    string:
        Cadena de texto con la tabla de multiplicar formateada línea por línea
    '''
    # Inicializamos una lista vacía para almacenar cada línea de la tabla
    tabla = []
    # Recorremos los números del 1 al 10 para calcular cada multiplicación
    for i in range(1, 11):
        # Calculamos el producto del número actual del bucle por el número dado
        resultado = i * number
        # Añadimos la línea formateada con la operación y el resultado a la lista
        tabla.append(f"{i} * {number} = {resultado}")

    # Unimos todas las líneas de la lista separadas por saltos de línea y devolvemos el resultado
    return "\n".join(tabla)

tablaMult = multi_table(5)
print(tablaMult)
```

```
ignguiltar@ORJ1:~/Escritorio/Python$ /usr/bin/python3 /home/ignguiltar/Escritorio/Python/actCodewars6.py
1 * 5 = 5
2 * 5 = 10
3 * 5 = 15
4 * 5 = 20
5 * 5 = 25
6 * 5 = 30
7 * 5 = 35
8 * 5 = 40
9 * 5 = 45
10 * 5 = 50
```

Actividad 7




```python
def cada_cons(lista, n):
    '''Crea subconjuntos en cascada de una lista de un tamaño n dado

    PARAMETERS
    ----------
    lista (list):
        Lista de elementos de la cual se extraerán los subconjuntos en cascada
    n (integer):
        Tamaño o longitud de cada subconjunto

    RETURNS
    -------
    list:
        Lista que contiene los subconjuntos en cascada (superpuestos) generados
    '''
    # Inicializamos una lista vacía para almacenar los subconjuntos en cascada
    resultado = []

    # Recorremos la lista hasta el índice donde aún se pueden formar subconjuntos de tamaño n
    for i in range(len(lista) - n + 1):
        # Extraemos la porción de tamaño n de la lista y la añadimos al resultado
```

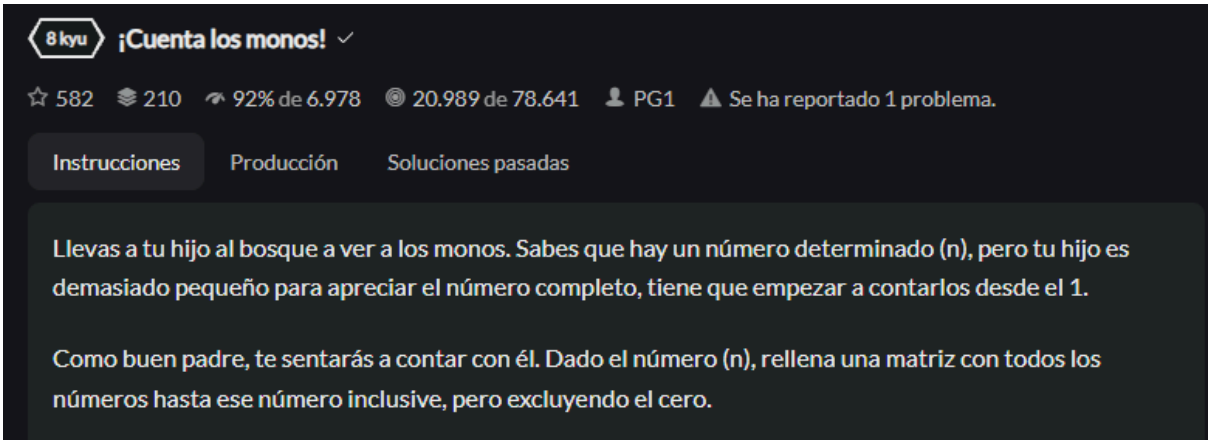
```
        resultado.append(lista[i:i+n])

    # Devolvemos la lista completa con todos los subconjuntos generados
    return resultado

print(cada_cons([1, 2, 3, 4], 2))
```

```
ignglitar@ORJ1:~/Escritorio/Python$ /usr/bin/python3 /home/ignglitar/Escritorio/Python/actCodewars7.py
[[1, 2], [2, 3], [3, 4]]
```

Actividad 8




```
def monkey_count(n):
    '''Genera una lista con los números desde el 1 hasta n para contar
los monos

    PARAMETERS
    ----------
    n (integer):
        Número total de monos en el bosque

    RETURNS
    -------
    list:
        Lista con los números desde el 1 hasta n inclusive
    '''
    # Inicializamos una lista vacía para almacenar los números de conteo
    resultado = []

    # Recorremos desde el 1 hasta n (inclusive)
    for i in range(1, n + 1):
        # Añadimos el número actual a la lista de conteo
```

```
        resultado.append(i)

    # Devolvemos la lista completa con el conteo de los monos
    return resultado

print(monkey_count(10))
```

```
ignglitar@ORJ1:~/Escritorio/Python$ /usr/bin/python3 /home/ignglitar/Escritorio/Python/actCodewars8.py
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Actividad 9




```
def remove_every_other(lista):
    '''Elimina cada segundo elemento de una lista, manteniendo el
primero y alternando

    PARAMETERS
    ----------
    lista (list):
        Lista de elementos de la cual se eliminarán elementos alternados

    RETURNS
    -------
    list:
        Lista resultante con los elementos conservados
    '''
    # Utilizamos el rebanado (slicing) de Python para seleccionar desde
el inicio hasta el final avanzando de 2 en 2 (índices 0, 2, 4, ...)
    # Esto mantiene automáticamente el primer elemento y elimina el
segundo de forma sucesiva
```

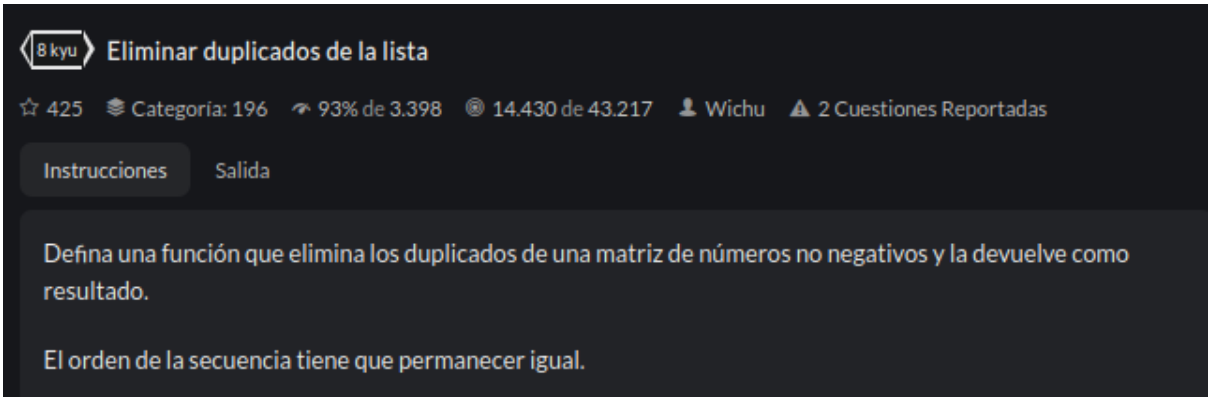
```
    resultado = lista[::2]

    # Devolvemos la nueva lista con los elementos filtrados
    return resultado

print(remove_every_other(["Nacho", "Eliminar", "Oscar", "Quitar",
"Borja"]))
```

```
ignguiltar@ORJ1:~/Escritorio/Python$ /usr/bin/python3 /home/ignguiltar/Escritorio/Python/actCodewars9.py
['Nacho', 'Oscar', 'Borja']
```

Actividad 10




```
def distinct(seq):
    '''Elimina los elementos duplicados de una lista de números
manteniendo el orden original

    PARAMETERS
    ----------
    seq (list):
        Lista de números no negativos que puede contener duplicados

    RETURNS
    -------
    list:
        Lista resultante sin duplicados y con el orden original
conservado
    '''
    # Convertimos la lista en un diccionario para eliminar duplicados
preservando el orden, y volvemos a transformarla en lista
    resultado = list(dict.fromkeys(seq))

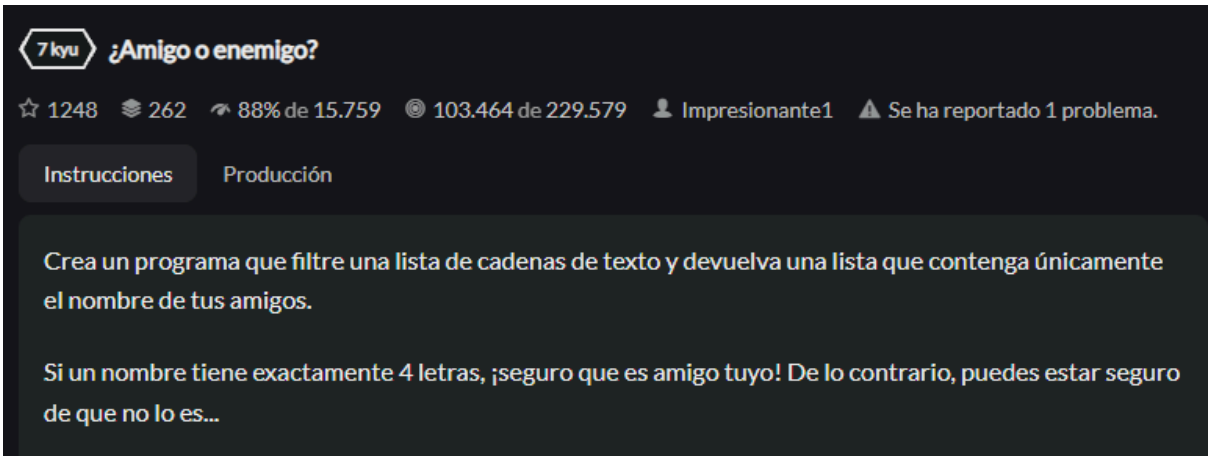
    # Devolvemos la lista limpia de duplicados
```

```
    return resultado


print(distinct([1, 2, 1, 1, 3, 2]))
```

```
ignglitar@ORJ1:~/Escritorio/Python$ /usr/bin/python3 /home/ignglitar/Escritorio/Python/actCodewars10.py
[1, 2, 3]
```

Actividad 11




```
def friend(x):
    '''Filtra una lista de cadenas de texto y devuelve únicamente los
nombres que tienen exactamente 4 letras

    PARAMETERS
    ----------
    x (list):
        Lista de cadenas de texto con nombres

    RETURNS
    -------
    list:
        Lista con los nombres que tienen exactamente 4 letras,
manteniendo el orden original
    '''
    # Inicializamos una lista vacía para almacenar los nombres de los
amigos (con 4 letras)
    resultado = []

    # Recorremos cada nombre de la lista de entrada
    for nombre in x:
```

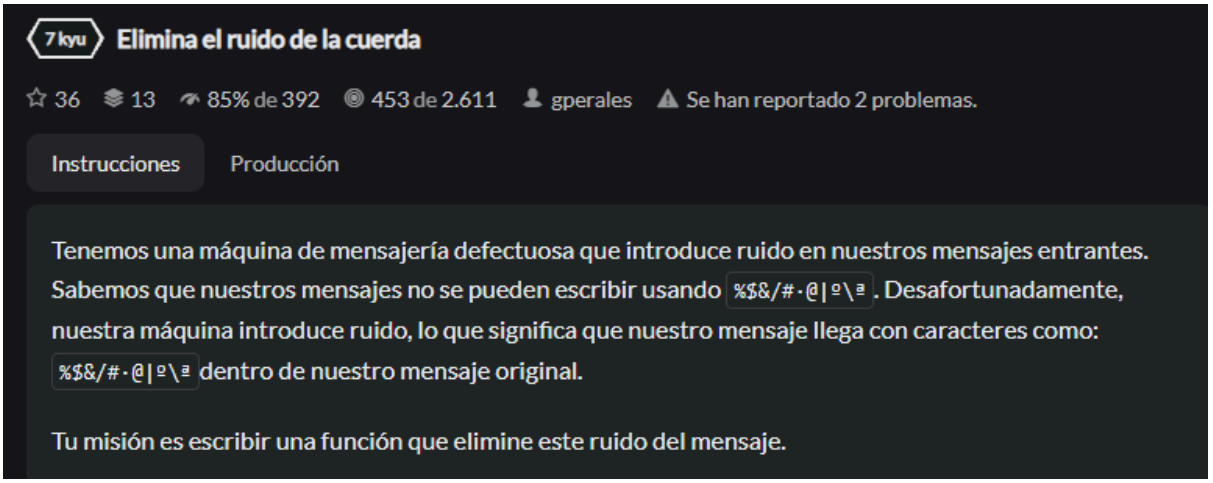
```
        # Comprobamos si la longitud del nombre es exactamente igual a
4
        if len(nombre) == 4:
            # Si cumple la condición, lo añadimos a la lista de
resultado
            resultado.append(nombre)

    # Devolvemos la lista filtrada con los amigos
    return resultado

print(friend(["Nacho", "Paco", "Oscar", "Manu"]))
print(friend(["Raul", "Pepe", "Jorge"]))
```

```
(.venv) PS C:\Users\Nacho\Desktop\SGE> & c:\Users\Nacho\Desktop\SGE\.venv\Scripts\
python.exe c:/Users/Nacho/Desktop/SGE/UD00_4.11/actCodewars11.py
['Paco', 'Manu']
['Raul', 'Pepe']
```

Actividad 12




```
def remove_noise(s):
    '''Elimina los caracteres de ruido específicos de un mensaje de
texto dado usando un bucle for

    PARAMETERS
    ----------
    s (string):
        Cadena de texto con ruido que se desea limpiar

    RETURNS
    -------
```

```
    string:
        Cadena de texto limpia sin los caracteres de ruido
    '''
    # Definimos la cadena que contiene todos los caracteres
considerados como ruido
    ruido = "%$&/#·@|°\\ª"

    # Inicializamos una cadena vacía para ir acumulando los caracteres
limpios
    resultado = ""

    # Recorremos cada letra (carácter) de la cadena de entrada
    for caracter in s:
        # Comprobamos si el carácter actual NO está dentro de los
caracteres de ruido
        if caracter not in ruido:
            # Si no es ruido, lo concatenamos al resultado
            resultado = resultado + caracter

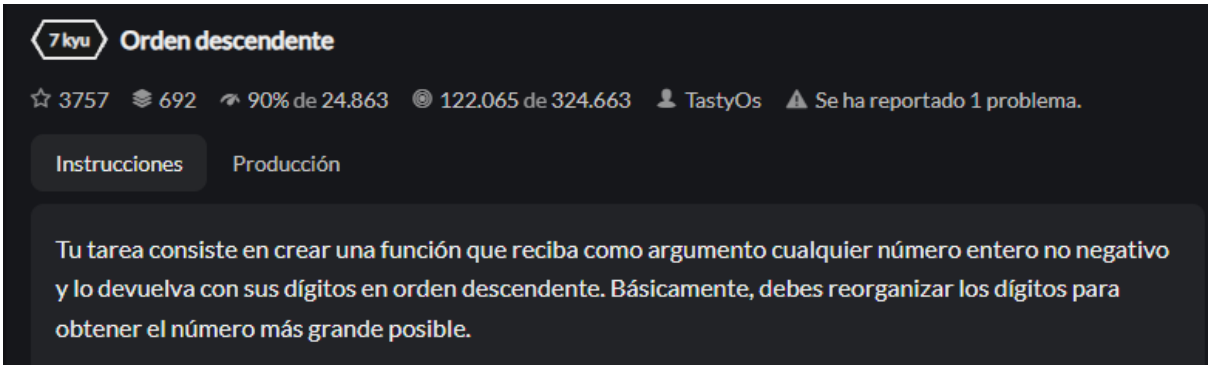
    # Devolvemos el mensaje limpio final
    return resultado

print(remove_noise("h%o&·%$·la s&%oy N&ach·%o"))
print(remove_noise("%$"))
print(remove_noise("abcd"))
```

```
(.venv) PS C:\Users\Nacho\Desktop\SGE>
hola soy Nacho

abcd
```

Actividad 13



```python
def descending_order(num):
    '''Reorganiza los dígitos de un número entero no negativo en orden descendente para obtener el número más grande posible

    PARAMETERS
    ----------
    num (integer):
        Número entero no negativo a reorganizar

    RETURNS
    -------
    integer:
        Número resultante con los dígitos ordenados de mayor a menor
    '''
    # Convertimos el número a una cadena de texto para poder manipular sus dígitos individualmente
    num_str = str(num)

    # Ordenamos los dígitos de la cadena en orden descendente (de mayor a menor)
    digitos_ordenados = sorted(num_str, reverse=True)

    # Unimos los caracteres ordenados en una sola cadena de texto
    resultado_str = "".join(digitos_ordenados)

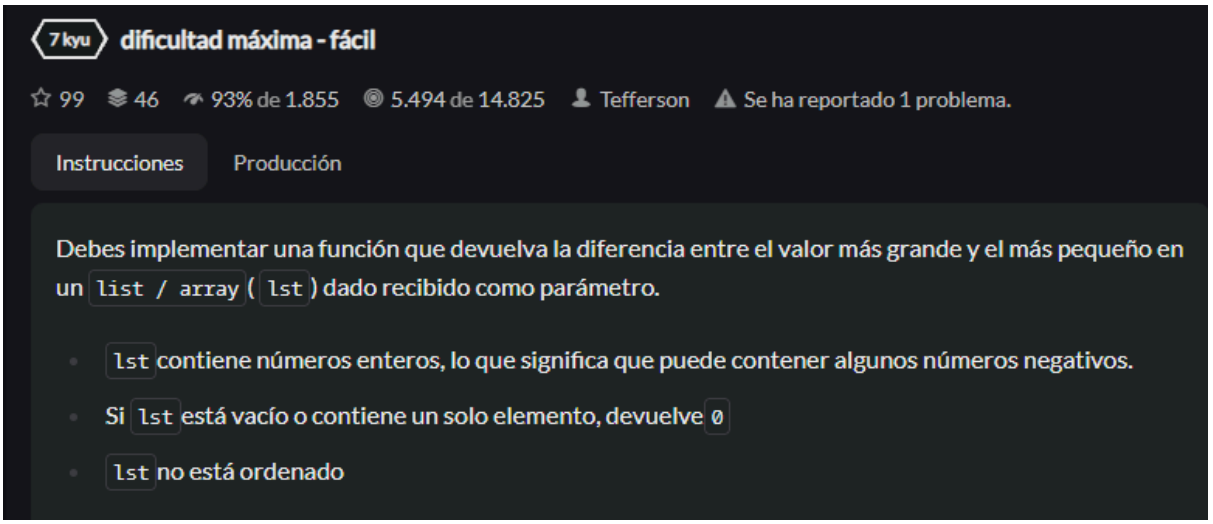
    # Convertimos la cadena de texto resultante de nuevo a un número entero
    resultado = int(resultado_str)

    # Devolvemos el número final con sus dígitos en orden descendente
    return resultado

print(descending_order(42145))
print(descending_order(145263))
print(descending_order(123456789))
```

```
(.venv) PS C:\Users\Nacho\Desktop\SGE> & c:\Users\Nacho\Desktop\SGE\.venv\Scripts\python.exe c:/Users/Nacho/Desktop/SGE/actCodewars13.py
54421
654321
987654321
```

Actividad 14




```python
def max_diff(lst):
    '''Calcula la diferencia entre el valor más grande y el más pequeño de una lista de números enteros

    PARAMETERS
    ----------
    lst (list):
        Lista de números enteros (puede incluir negativos y no está ordenada)

    RETURNS
    -------
    integer:
        Diferencia entre el valor máximo y el mínimo; 0 si la lista está vacía o tiene un solo elemento
    '''
    # Comprobamos si la lista está vacía o tiene un solo elemento
    if not lst or len(lst) < 2:
        # Devolvemos 0 tal como se especifica en las condiciones
        return 0

    # Encontramos el número más grande de la lista
    max_val = max(lst)

    # Encontramos el número más pequeño de la lista
    min_val = min(lst)

    # Calculamos la diferencia restando el mínimo al máximo
```

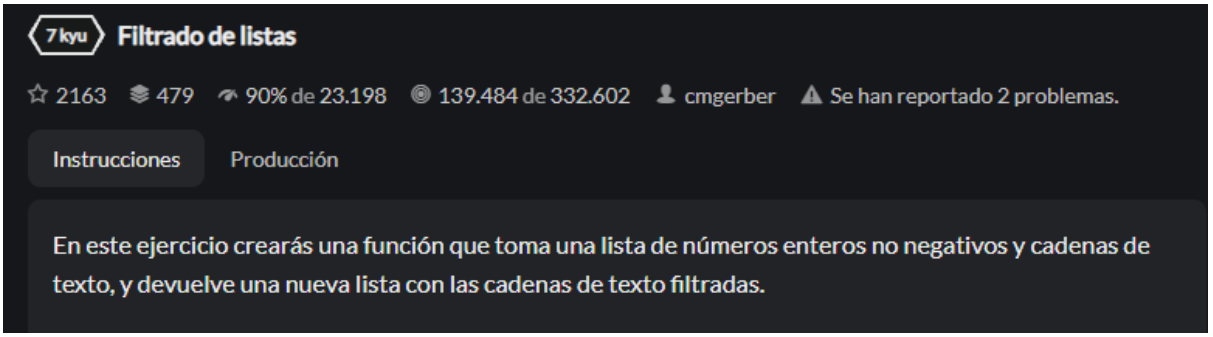
```
    resultado = max_val - min_val

    # Devolvemos el resultado final
    return resultado

print(max_diff([1, 2, 3, 4]))
print(max_diff([1, 2, 3, -4]))
print(max_diff([5]))
print(max_diff([]))
```

```
(.venv) PS C:\Users\Nacho\Desktop\SGE> & c:\Users\Nacho\Desktop\SGE\.venv\Scripts\python.exe
c:/Users/Nacho/Desktop/SGE/actCodewars14.py
3
7
0
0
```

Actividad 15




```
def filter_list(lst):
    '''Filtra una lista mixta de números enteros y cadenas de texto,
devolviendo únicamente los números enteros

    PARAMETERS
    ----------
    lst (list):
        Lista que contiene una mezcla de números enteros no negativos y
cadenas de texto

    RETURNS
    -------
    list:
        Lista resultante que contiene solo los números enteros,
manteniendo el orden original
    '''
```

```python
    # Inicializamos una lista vacía para almacenar únicamente los
números enteros
    resultado = []

    # Recorremos cada elemento de la lista de entrada
    for elemento in lst:
        # Comprobamos si el tipo del elemento es un número entero
(type() == int)
        if type(elemento) == int:
            # Si es un número entero, lo añadimos a la lista de
resultado
            resultado.append(elemento)

    # Devolvemos la lista filtrada solo con los enteros
    return resultado

print(filter_list([1, 2, 'a', 'b']))
print(filter_list([1, 'a', 'b', 0, 15]))
print(filter_list([1, 2, 'aasf', '1', '123', 123]))
```

```
(.venv) PS C:\Users\Nacho\Desktop\SGE> & c:\Users\Nacho\Desktop\SGE\.venv\Scripts\python.exe
c:/Users/Nacho/Desktop/SGE/actCodewars15.py
[1, 2]
[1, 0, 15]
[1, 2, 123]
```